Andy, Yu-Guang Chen

EE6094 CAD for VLSI Design

Programming Assignment 3: Graph-based STA

(Due: 23:59:59, 2025/05/15)

Introduction

In modern digital circuit design, ensuring that signals propagate through a circuit within specified timing constraints is critical for functionality and performance. Static Timing Analysis (STA) is a fundamental technique used to verify the timing behavior of digital circuits without the need for exhaustive simulation. This assignment focuses on implementing a Graph-based Static Timing Analysis (STA) engine, which will allow you to analyze the timing of a digital circuit by modeling it as a directed acyclic graph (DAG) and propagating timing information through its nodes.

In this assignment, you will:

1. Model a digital circuit as a timing graph: Represent the circuit's components (gates and interconnects) as nodes and edges in a graph.
2. Propagate arrival and required times: Calculate the earliest and latest signal arrival times at each node to determine if the circuit meets its timing constraints.

Background

Static Timing Analysis (STA) is a fundamental technique in modern digital circuit design, essential for verifying the timing behavior of circuits without the need for exhaustive simulation. Unlike dynamic timing analysis, which relies on test vectors and simulation, STA analyzes the circuit's structure to determine whether it can operate correctly under worst-case timing conditions. This makes STA highly scalable and efficient, particularly for large-scale designs such as microprocessors and system-on-chip (SoC) designs.

STA operates by modeling the circuit as a directed acyclic graph (DAG), where nodes represent components like gates and flip-flops, and edges represent signal propagation paths. Key concepts in STA include arrival time, required time, and slack, which are used to evaluate whether the circuit meets its timing constraints. However, STA is inherently conservative, often leading to pessimism due to its worst-case analysis approach. Challenges such as false paths, on-chip variations (process, voltage, and temperature), and the exponential growth of paths in large designs further complicate STA. Despite these challenges, STA remains the industry standard for timing verification, enabling designers to optimize circuits for performance and reliability while ensuring timing closure. Future directions for STA include multi-corner multi-mode analysis, signal integrity considerations, and statistical timing analysis to address the increasing variability in modern IC designs.

Problem Definition

Given a gate-level netlist and a cell library, find the longest and the shortest delay and their corresponding path under given input pattern.

There are 3 steps to follow:

Step1. Build the delay graph according to the given netlist.

Step2. Calculate the delay of each instance in the graph following topological order.

Step3. Find the longest and the shortest delay and their corresponding path.

Input file format

1. Flattened gate-level Verilog netlist (.v)

This file follows the standard Verilog gate-level syntax. To simplify the problem, the given netlist is flattened, i.e., only one module exists in the file. Also, only three kinds of cells (NAND2, NOR2, INV) will appear in this file. Sequential circuits are not necessary to be considered.

Your program should be able to deal with extra space (or TAB), extra new lines, block comments (/**/), and the comment lines starting with double slash (//) in this netlist file. Below is a simple example. Please note that the delay values in this example are just assumptions to simplify the calculation. Actual delay should be obtained from the given library information.

```
module c17 (N1, N2, N3, N6, N7, N22, N23);
    input N1, N2, N3, N6, N7;
    output N22, N23;
    //internal wires
    wire n8, n9, n10, n11, n12;
    /*
        */
    NOR2X1 U8 (.A1(n8), .A2(n9), .ZN(N23));
    NOR2X1 U9 (.A1(N2), .A2(N7), .ZN(n9));
    INVX1 U10 (.I(n10), .ZN(n8)); //invertor
    NANDX1 U11 (.A1(n11), .A2(n12), .ZN(N22));
    NANDX1 U12 (.A1(N2), .A2(n10), .ZN(n12));
    NANDX1 U13 (.A1(N6), .A2(N3), .ZN(n10));
    NANDX1 U14 (.A1(N1), .A2(N3), .ZN(n11));
endmodule
```

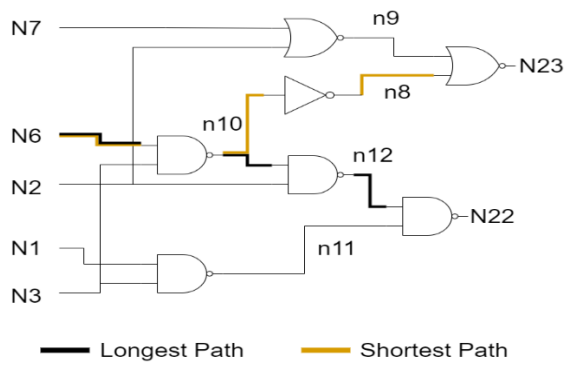

Fig 1. An example of the gate-level netlist and its corresponding description

2. Simplified delay models (test_lib.lib)

This file is simplified from the standard liberty format, which collects the timing and power information for each cell. The default units are also given in the file, which are ns (timing), pF (capacitance), V (voltage), and mA (current). To simplify the problem, we assume that the timing and power information of the same output is the same for all input path, i.e., A1->ZN and A2->ZN have the same delay and power.

Therefore, only one table is recorded for each output. The table index is defined in *lu_table_template*, which include total output loading and input transition time. Please note that the total output loading and input transition time should be calculated based on the real circuit connection. The input capacitance of a cell is the output loading of the preceding cell. The output rising/falling time of a cell will be the input transition time of the succeeding cells.

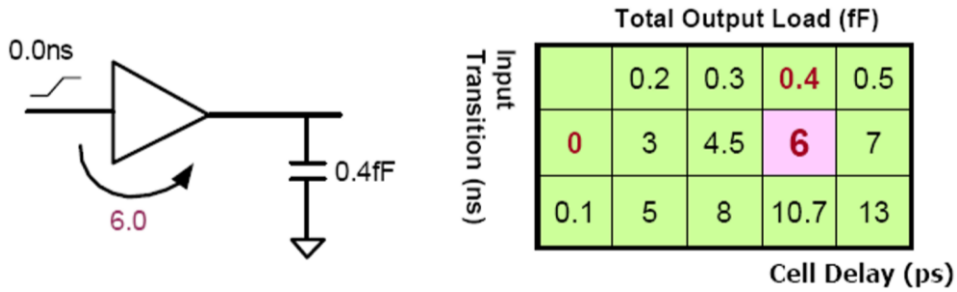

Fig 2. An example of Gate Delay Models

Output file format

After executing your program, three output files should be generated for each step. You must follow the naming rules: Load_*StudID_Casename*.txt, Delay_*StudID_Casename*.txt and Path_*StudID_Casename*.txt, respectively. For example, if the netlist file is named c17.v, the *Casename* would be c17.

Step1. Calculate output loading of each instance

Construct the circuit graph according to the given netlist. In this step, we proceed to verify the correctness of the graph construction by checking the output loading of each cell. The output loading of each cell would be the summation of the input capacitance of all the fanout cells. For the primary output in the given netlist, please set the output loading as **0.03pF**.

Sort your results in descending order and name the file as **"Load_*StudID_Casename*.txt"**. If the output loadings are identical, sort them according to instance number in ascending order. Besides, you must print each value to the 6th decimal places.

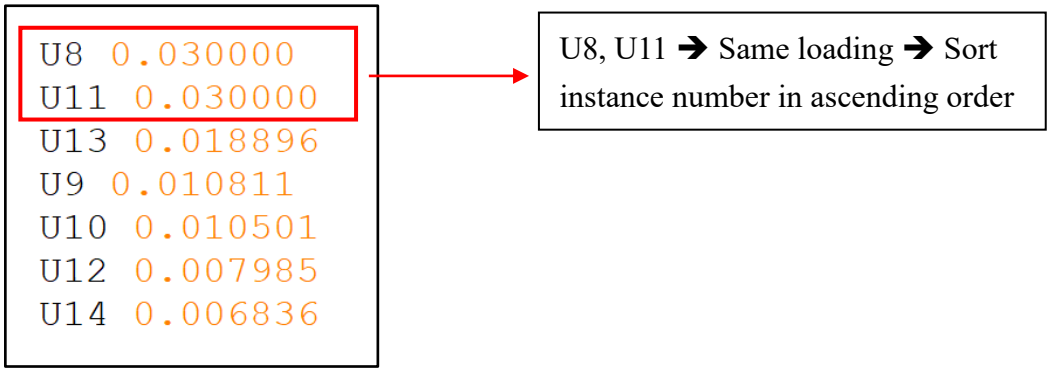

Fig 3. An example of Load_*StudID_Casename*.txt

Step2. Calculate propagation delay, and output transition time of each instance

Before finding the worst-case delay path for the whole circuit, you must determine the propagation delay and output transition time of each instance based on the circuit connection.

Input transition time is defined by the input that arrives latest if the instance has multiple inputs. Propagation delay and output transition time of the instance is then defined by the worst-case output, i.e., you need to calculate both output situations (output = 0 & 1), the one that has larger cell propagation delay will be the worst-case output.

If the output is 0, output transition time is defined by its output falling time. If the output is 1, output transition time is defined by its output rising time.

In this assignment, you don't need to consider the correctness of gate logic, just find the output that produces larger propagation delay.

To verify the results of this step, please report the worst-case output and timing information of each instance in the following format. Each row shows an instance name, worst-case output value, corresponding worst-case delay, and output transition time in order. Also, we assume that each wire between any two instances has **0.005ns delay**.

Sort your results according to delay value in descending order and name the file as **"Delay_*StudID_Casename*.txt"**. If the delay values are identical, sort them according to instance number in ascending order. Besides, you must print value of propagation delay and output transition time to the 6th decimal places.

```
U11 1 0.070545 0.116993
U8 1 0.064212 0.103237
U12 1 0.044436 0.059305
U10 0 0.035620 0.042872
U13 1 0.033812 0.054006
U9 1 0.028109 0.053694
U14 1 0.024308 0.025355
```

Fig 4. An example of Delay_*StudID_Casename*.txt

Step3. Find the longest and the shortest delay from all output paths

According to the delay information, compare the worst-case delay of all outputs. The largest one is the longest delay, and the smallest one is the shortest delay. Report them in different lines. To simplify the results, only nets' name is required to be reported, instances' name is not required. You must output the longest and the shortest delays and their corresponding delay path based on the given library file **"test_lib.lib"**. If the instance has multiple input signals that has same arrival time, you can output either one of them. Name the

output file as **"Path_*StudID_Casename*.txt"**. Also, you must print the value of the path delay to the 6th decimal places.

```
Longest delay = 0.158793, the path is: N6 -> n10 -> n12 -> N22
Shortest delay = 0.143645, the path is: N6 -> n10 -> n8 -> N23
```

Fig 5. An example of Path_*StudID_Casename*.txt

Requirement

1. You have to write this program in C or C++. No open-source codes are allowed to be used. (i.e., you MUST implement the tool by yourself). You can use any data structure to realize your program.

2. You must submit a **single .tar file** named *StudID*_PA3.tar (e.g., 9862534_PA3.tar). The directory structure inside should follow this format:

```
9862534_PA3/                  (Project folder)
├── 9862534_PA3.cpp           (Main source file)
├── 9862534_PA3_report.pdf    (Report file - only .pdf is accepted)
├── inc/                      (Header files, if needed)
│   ├── file1.h
│   ├── file2.h
├── src/                      (Source files, if needed)
│   ├── file1.cpp
│   ├── file2.cpp
```

Fig 6. PA3 folder structure example

   (1) Incorrect naming results in a 10% penalty.

   (2) Only .pdf reports are accepted; .doc or .docx will not be graded.

   (3) No duplicate file extensions (e.g., 9862534_PA3.cpp.cpp is incorrect).

   (4) Including the testcase files in a 10% penalty.

3. Use the following command in a Linux environment to compress the folder and create the *.tar* file, which should be uploaded to ee-class.

$tar cvf *StudID*_PA3.tar *StudID*_PA3

4. Compilation & Execution Command
   (1) Compile: This command compiles your code and generates the executable file

   $make all

   (2) Execute: This command runs the executable with the provided input file and the required library. run: ➔ ./$(TARGET) $(input) test_lib.lib

   (3) Clean: This command will automatically remove all the objects and executable file

generated by make all.

```
$make clean
```

5. We don't restrict the report format and length. In your report, you have to include at least:
   (1) How to compile and execute your program; (You can use screenshot(s) to explain)
   (2) The completion of the assignment; (If you complete all requirements, specify all)
   (3) The hardness of this assignment and how you overcome it;
   (4) Do you have any suggestions about this programming assignment?

Grading

The grading is as follows:

1. Correctness of your code: 40%
   (1) Step1: 15%
   (2) Step2: 15%
   (3) Step3: 10%
2. Performance: 20%
   (1) You must pass the correctness test to be evaluated.
   (2) The evaluation criteria are based on the program execution time.
3. Readability of your code: 10%
4. The report: 10%
5. Demo session: 20%

Please submit your assignment on time. Otherwise, the penalty rule will apply:

1. Within 24hrs delay: 20% off
2. Within 48hrs delay: 40% off
3. More than 48hrs: 0 point

Contact

For all questions about PA3, please ask on PA3 discussion forum or send email to TA 丁慧慈 (dinghc99@gmail.com) with the title [CAD PA3].